

Tech Challenge - Fases 1, 2 e 3 -

Documento de entrega (back-end Oficina)

Identificação do grupo

- **Nome do grupo:** Oficina Turbo
- **Código/identificador interno:** Grupo 106

Participantes (nome + Discord)

- **Felipe Ricarte Magalhães** - Discord: **@felipe.ricarte**

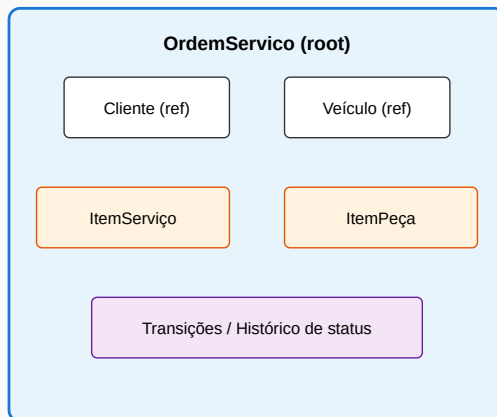
Links do projeto

- **Repositório (privado):** <https://github.com/ricartefelipe/oficina-springboot-mvp>
 - **Branches (nomes no Git):** main (estável), develop (integração).
 - **Acesso ao avaliador SOAT:** usuário ou equipe **soat-architecture** com permissão de leitura (ou conforme o enunciado). Passos: Convidar o soat-architecture no GitHub.
- **Documento para o PDF do portal (Fase 2):** entrega-portal-fase2.md - estrutura alinhada ao enunciário (repo, arquitetura, vídeo, Swagger).
- **Documento para o PDF do portal (Fase 3):** entrega-portal-fase3.md - links dos 4 repositórios, CI/CD, documentação e confirmação de acesso.
- **Swagger (local):** <http://localhost:8080/api/swagger-ui/index.html>
- **Vídeo demonstrativo (≤ 15 min):** o URL público (YouTube ou Vimeo) será o mesmo indicado na tabela **Links rápidos (APIs e vídeo)** do README quando estiver publicado.
- **PDF deste documento (gerado):** `submission.pdf` (na mesma pasta; ver Conversão para PDF).

Diagramas de arquitetura (DDD)

Artefatos de texto: [índice DDD](#), [Domain Storytelling](#), [Dicionário linguagem ubíqua](#).

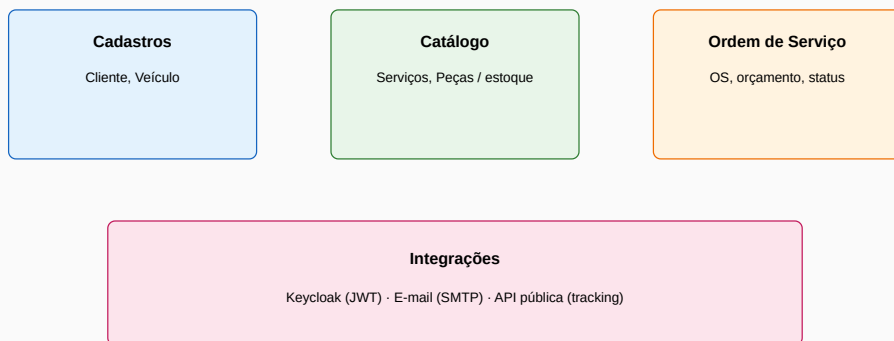
Agregado: Ordem de Serviço



Invariantes no domínio; API e persistência via adaptadores

Diagrama - agregado Ordem de Serviço

Event Storming (resumo) — Oficina



Ver também event-storming.md e diagramas.md (Mermaid)

Diagrama - event storming (contextos)

 Diagrama - event storming lousa (elementos C/A/E/P/R)

Diagrama - event storming lousa (elementos C/A/E/P/R)

Checklist entrega Fase 2

Estes passos dependem da sua conta no GitHub, do portal da disciplina e da gravação do vídeo.

1. Convidar soat-architecture no repositório

1. Abre <https://github.com/ricartefelipe/oficina-springboot-mvp>
2. **Settings** → **Collaborators and teams** (ou **Manage access**)
3. **Add people / Invite a collaborator**
4. Pesquisa **soat-architecture** (usuário ou equipe indicada no enunciado) e envia convite com permissão adequada (normalmente **Read** para revisão).
5. Confirma no e-mail/GitHub que o convite foi aceite (se aplicável).

2. PDF no portal da disciplina

1. Gera o PDF deste documento (seção Conversão para PDF).
2. Inclui no PDF (ou anexos) os **diagramas DDD** referidos em [docs/ddd/diagramas.md](#) e os SVG em [docs/ddd/diagrams/](#) se o enunciado pedir evidência visual.
3. Faça o upload no **portal SOAT** no prazo indicado na disciplina (envio manual pelo aluno).

3. Vídeo (≤ 15 minutos)

Roteiro e tópicos obrigatórios: [docs/video-script.md](#). Após publicar no **YouTube** ou **Vimeo**, atualize o URL no [README](#) (tabela **Links rápidos**); o documento de portal [entrega-portal-fase2.md](#) deve refletir o mesmo link na respectiva seção.

Checklist entrega Fase 3

1. Confirmar os quatro repositórios separados:
 - oficina-auth-lambda
 - oficina-infra-kubernetes-
 - oficina-infra-database
 - oficina-app
 2. Confirmar main protegida e merge por PR em todos os repositórios.
 3. Confirmar usuário soat-architecture com acesso de leitura em todos.
 4. Atualizar e gerar PDF de [entrega-portal-fase3.md](#).
 5. Preencher/atualizar link do vídeo (quando disponível) antes do envio no portal.
-

Resumo do entregável

Fase 1 (MVP)

- Artefatos DDD adicionais: Domain Storytelling, Dicionário de linguagem ubíqua, Event Storming com figuras SVG (incl. lousa com elementos e agregados).
- CRUDs administrativos: clientes, veículos, serviços, peças/insumos (estoque)
- Ordens de serviço: criação, orçamento automático, transições de estado, listagem e detalhe
- Consulta pública por `trackingCode` e aprovação de orçamento com validações (CPF/CNPJ)
- Segurança: JWT (Keycloak, role ADMIN) nos endpoints admin
- Métrica: tempo médio de execução (`EM_EXECUCAO` → `FINALIZADA`)
- Dockerfile + Docker Compose
- Testes (unitários e integração) + JaCoCo
- Documentação DDD (Event Storming, diagramas, linguagem ubíqua) + roteiro de vídeo

Fase 2 (resiliência e escalabilidade)

- Arquitetura em evolução (hexagonal onde aplicável), **modelos de domínio sem JPA** (persistência em entidades e mappers na infra), testes nos fluxos críticos
- **Git Flow**: integração em develop, releases via PR para main — ver docs/development/gitflow.md
- **Kubernetes**: manifestos em /k8s (Deployment, Service, HPA, probes, etc.; `secret.example.yaml` e `secret.placeholder.yaml`)
- **IaC**: Terraform em /infra (rede AWS; **RDS PostgreSQL opcional** via `enable_rds`)
- **CI/CD**: GitHub Actions (Maven, Terraform validate, imagem **GHCR**; workflows manuais para deploy em cluster e Terraform na AWS)
- Diagramas DDD versionados (SVG em `docs/ddd/diagrams/`)

Fase 3 (operação corporativa)

- **4 repositórios** com CI/CD separados (Lambda, infra K8s, infra BD, app).
- **API Gateway + Serverless** para autenticação CPF/JWT (repo `oficina-auth-lambda`).
- **Banco gerenciado e Terraform** (repo `oficina-infra-database`).
- **Cluster Kubernetes + Terraform** (repo `oficina-infra-kubernetes`).

- **Aplicação principal em Kubernetes** com pipelines de CI e deploy por branch (develop/main) no repo oficina-app.
- **Arquitetura e documentação:** ADRs, RFC, diagramas, guia Fase 3 em docs/fase3/.

Relatório de vulnerabilidades (scan)

- **Arquivo do relatório:** docs/security/vulnerability-report.md
- **Evidências geradas pelo scan:** build/security/*
- **Resumo (valores de exemplo):**
 - CRITICAL: 0
 - HIGH: 0
 - MEDIUM: 2
 - LOW: 6
- **Plano de mitigação:** descrito no relatório (upgrade de dependências, ajustes de configuração e revisão contínua no pipeline).

Como executar localmente (para o avaliador)

Pré-requisito: Docker + Docker Compose v2

1. Subir o ambiente:

```
docker compose up --build
```

2. Obter token ADMIN (Keycloak):

```
export TOKEN="$(./scripts/get-admin-token.sh)"
```

3. Validar endpoints (exemplo):

```
curl -i http://localhost:8080/api/admin/clientes \
-H "Authorization: Bearer ${TOKEN}"
```

4. Rodar testes e cobertura:

```
mvn verify
```

5. Rodar scan de vulnerabilidades e gerar evidências:

```
./scripts/security/run-security-scans.sh
```

Conversão para PDF (offline)

Recomendado (Windows, PDF com diagramas embutidos): na raiz do repositório:

```
.\scripts\delivery\md-to-pdf-edge.ps1 -InputMd  
  "docs\delivery\submission.md"
```

Gera docs/delivery/submission.pdf (e um .html intermédio, ignorado pelo Git).

Com Pandoc só (sem diagramas rasterizados como acima):

```
pandoc docs/delivery/submission.md -o docs/delivery/submission.pdf
```

Sem Pandoc no Windows: winget install JohnMacFarlane.Pandoc, ou abrir este .md num editor com pré-visualização Markdown e usar **Imprimir** → **Salvar como PDF**.